# Data Structures & Algorithms

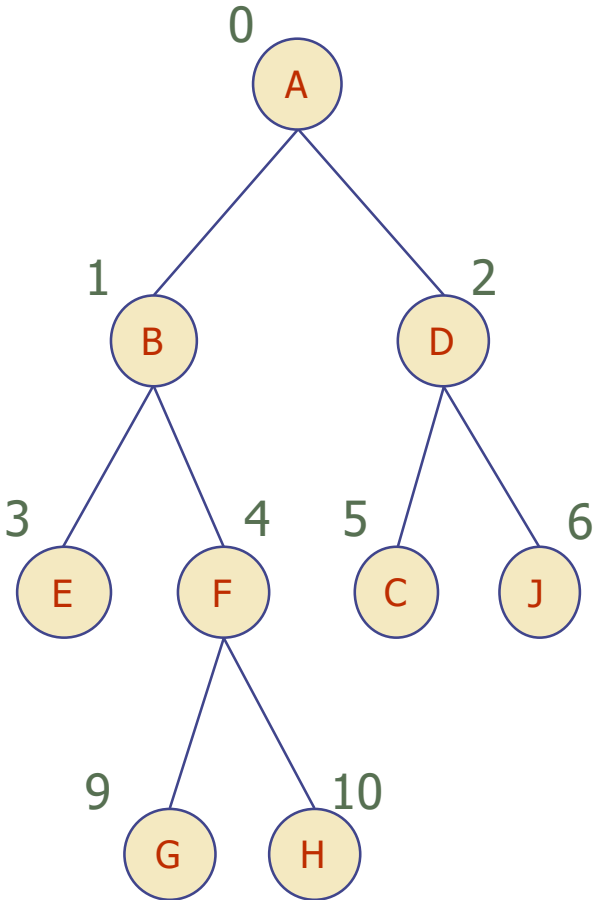
Tree Representations

# Tree Representations

- Possible ways to represent trees:
  - Array-based (for binary trees)
  - Linked

# Array-Based Representation for Binary Trees

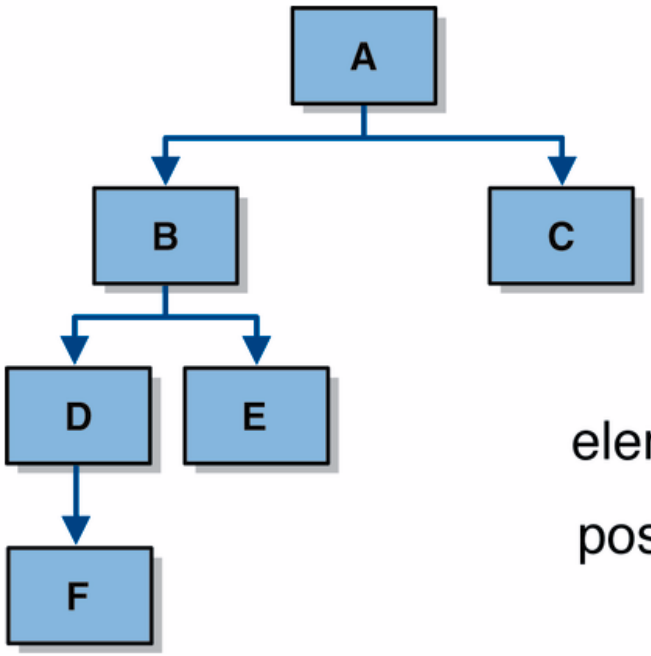
- Binary nodes are stored in an array *A*




Node **v** is stored at A[**rank(v)**]

- rank(root) = 0
- rank(left) = 2 · rank(parent(left)) + 1
- rank(right) = 2 · rank(parent(right)) + 2

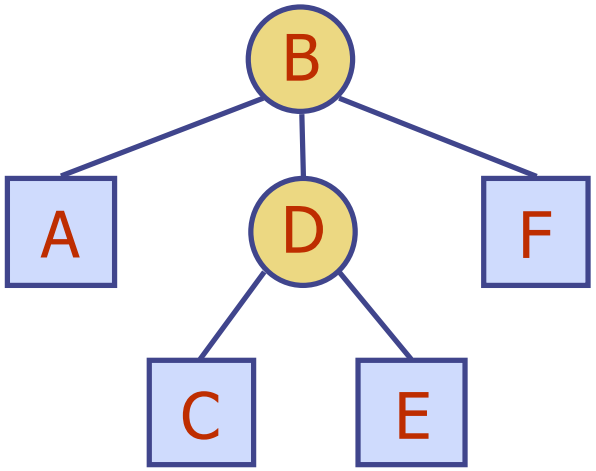
# Array-Based Representation: Issues?

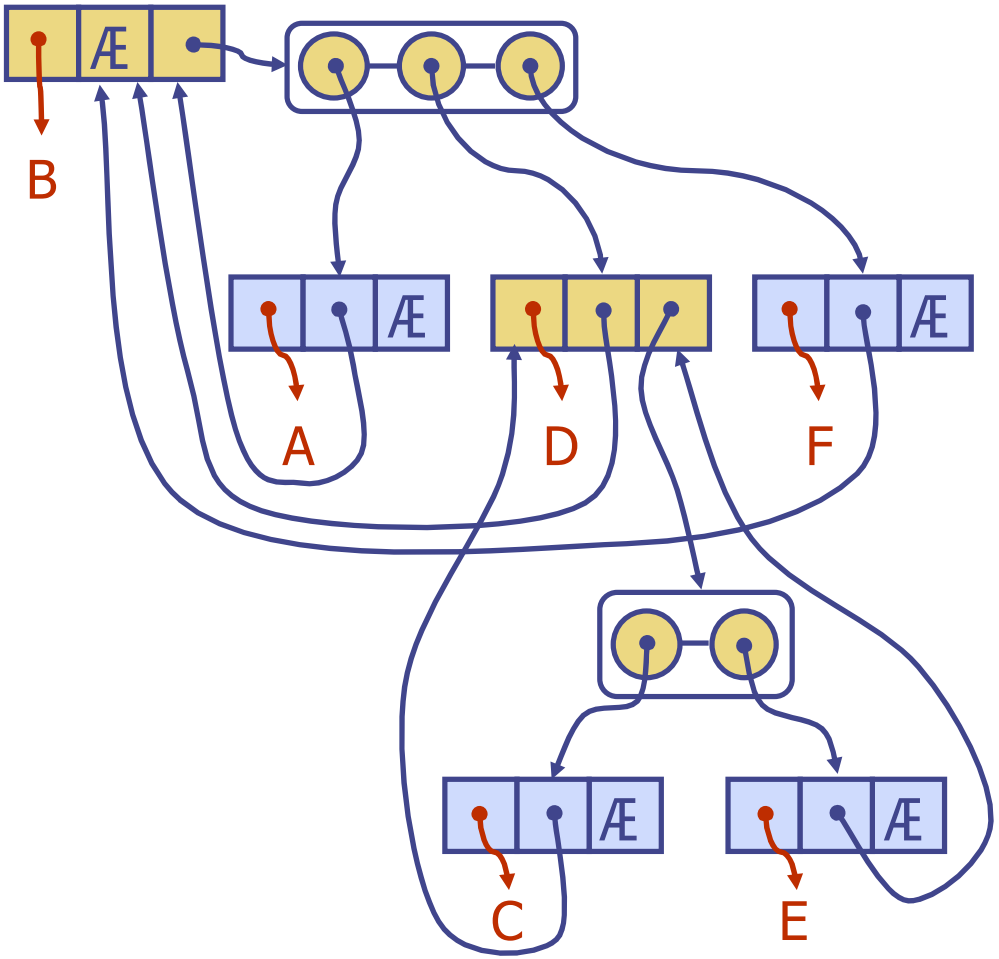



| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| element | A | B | C | D | E | | | F |
| position | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

# Linked Representation

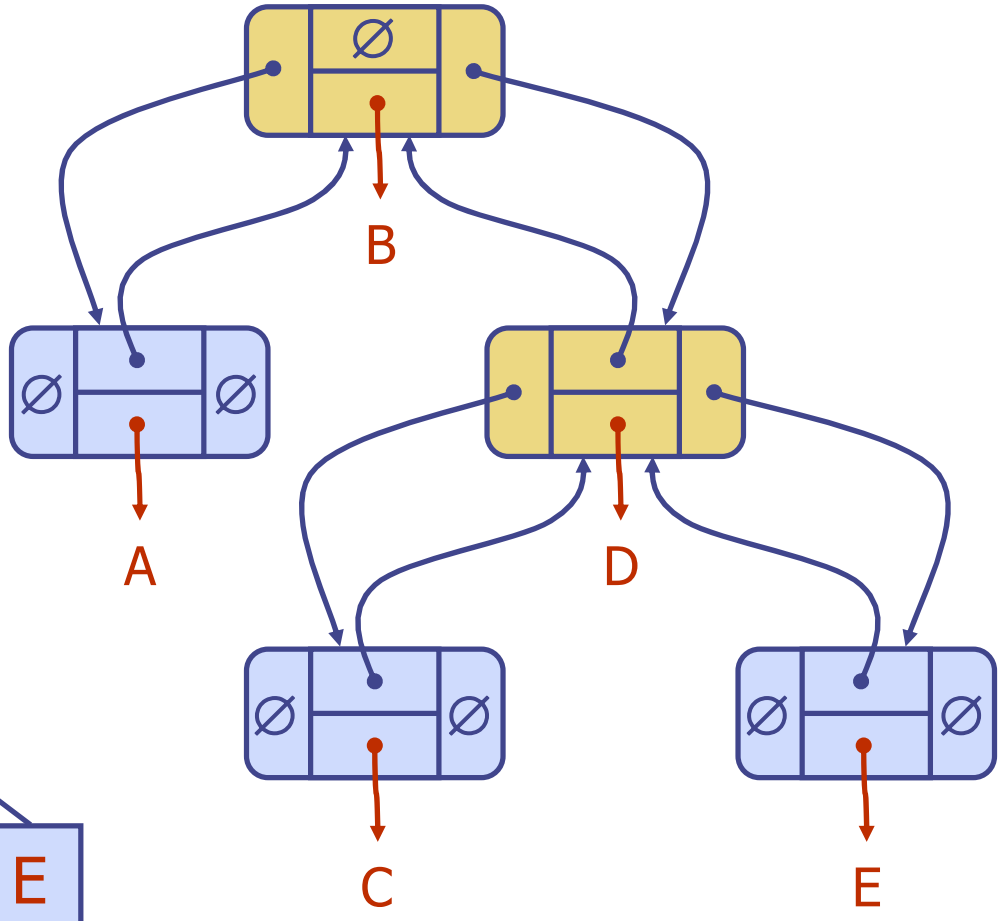
- Nodes are represented by objects storing:
  - Element
  - Parent node
  - List of child nodes

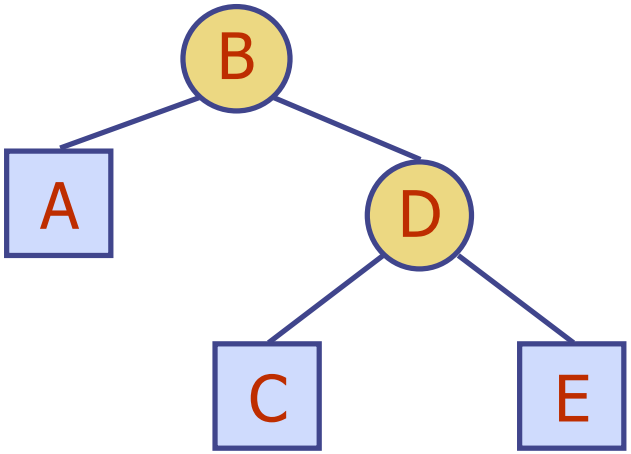


# Linked representation for binary trees

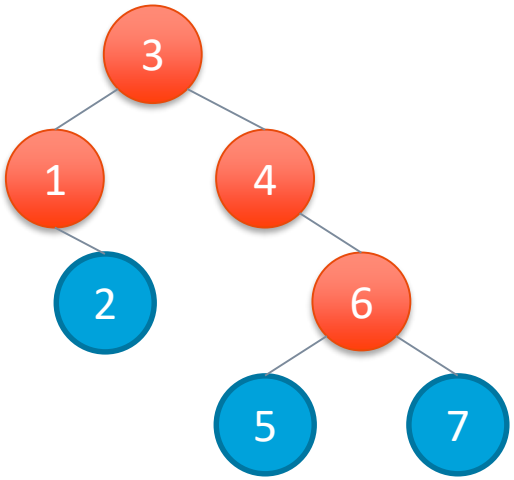
- Nodes are represented by objects storing:
  - Element
  - Parent node
  - Left child
  - Right child



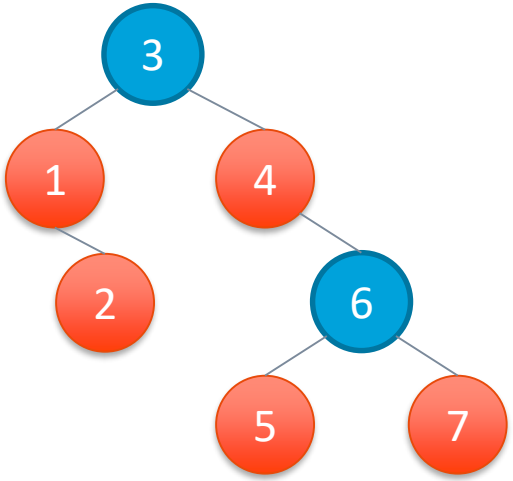
# Removing Nodes

- How to remove elements
  - Leafs



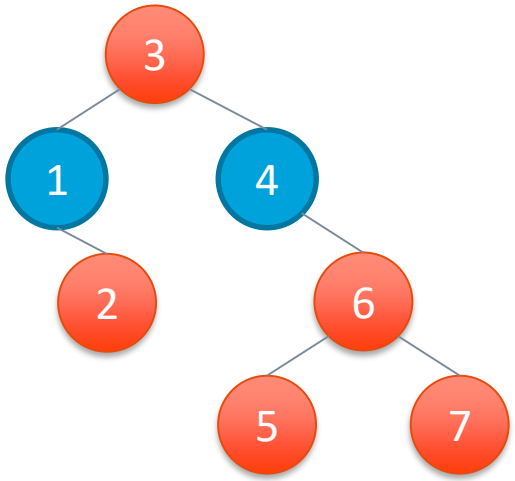
# Removing Nodes

- How to remove elements
  - Leafs
  - Two children
    - Not possible



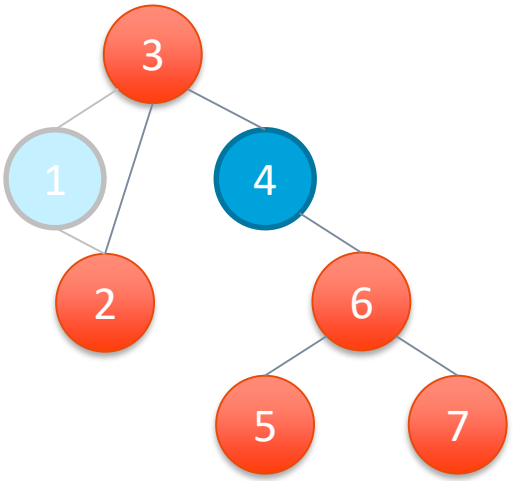
# Removing Nodes

- How to remove elements
  - Leafs
  - Two children
  - One child
    - Replace node by child



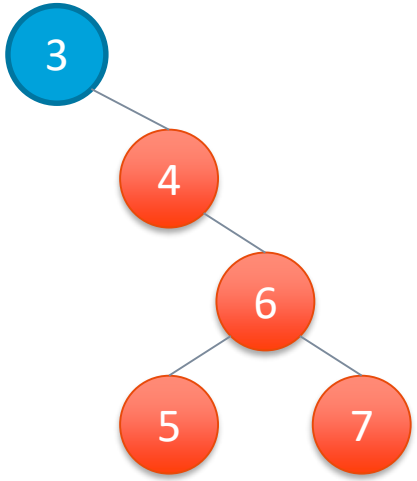
# Removing Nodes

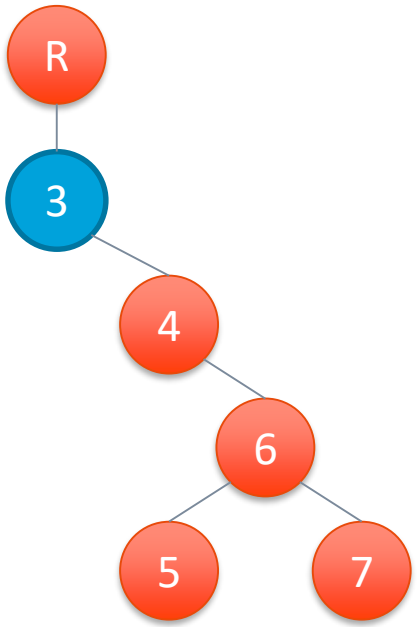
- How to remove elements
  - Leafs
  - Two children
  - One child
    - Replace node by child



# Removing Nodes

- How to remove elements
  - Leafs
  - Two children
  - One child
  - Root
    - How to make a different node root?



# Removing Nodes

- How to remove elements
  - Leafs
  - Two children
  - One child
  - Root
    - How to make a different node root?
    - Introduce  additional pseudo node